MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri

Proje Raporu

Sultan Bacakoğlu
22290189

Github Linki

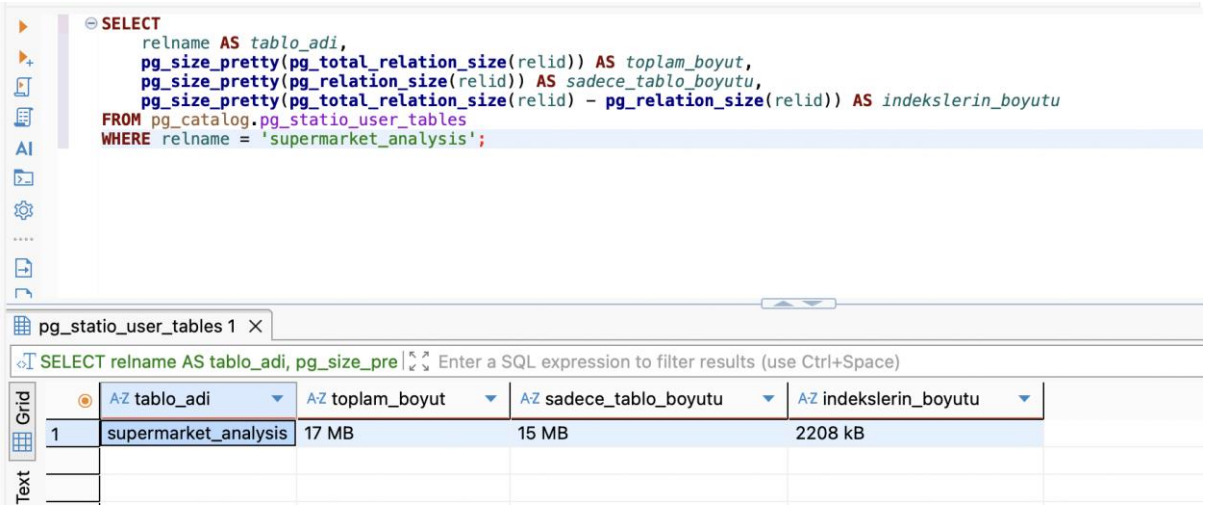
İÇİNDEKİLER

VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME RAPORU 2
1. Veri Yoğunluğunun Yönetimi ve Disk Alanı Analizi 3
2. Veritabanı İzleme (İstatistik ve Performans Takibi) 3
3. Sorgu İyileştirme ve Uzun Süren Sorguların Optimizasyonu 4
4. İndeks Yönetimi (Gereksiz İndekslerin Kaldırılması) 4
5. Veri Yöneticisi Rolleri (Erişim Yönetimi) 6

VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME RAPORU

1. Veri Yoğunluğunun Yönetimi ve Disk Alanı Analizi

Proje kapsamında öncelikle performans testlerinin gerçekçi senaryolarda yapılabilmesi için veri yoğunluğu artırılmıştır. Orijinal veri seti 100 kat büyütülerek tablonun hacmi genişletilmiştir. Artan veri yoğunluğu sonrasında tablonun ve performans için oluşturulması planlanan indekslerin disk üzerinde kapladığı alan, PostgreSQL sistem fonksiyonları (pg_size_pretty) kullanılarak analiz edilmiş ve disk kapasitesi planlaması yapılmıştır.



```sql
SELECT
    relname AS tablo_adi,
    pg_size_pretty(pg_total_relation_size(relid)) AS toplam_boyut,
    pg_size_pretty(pg_relation_size(relid)) AS sadece_tablo_boyutu,
    pg_size_pretty(pg_total_relation_size(relid) - pg_relation_size(relid)) AS indekslerin_boyutu
FROM pg_catalog.pg_statio_user_tables
WHERE relname = 'supermarket_analysis';
```

pg_statio_user_tables 1

SELECT relname AS tablo_adi, pg_size_pre | Enter a SQL expression to filter results (use Ctrl+Space)

	tablo_adi	toplam_boyut	sadece_tablo_boyutu	indekslerin_boyutu
1	supermarket_analysis	17 MB	15 MB	2208 kB

2. Veritabanı İzleme (İstatistik ve Performans Takibi)

Veritabanı performansını arka planda izlemek amacıyla, sistem istatistik görünümleri (pg_stat_user_tables) kullanılmıştır. Yapılan analizde, tablo üzerinde gerçekleştirilen sorguların sistemi ne kadar yorduğu izlenmiştir. İlk bulgulara göre sistemin akıllı arama yapmak yerine tüm tabloyu baştan sona taradığı (Sequential Scan) tespit edilmiştir.

```
SELECT * FROM supermarket_analysis WHERE siparis_id + 0 = 85432;
```

Execution plan - 1 | Execution plan - 2

Node Type	Entity	Cost	Rows	Time	Condition
Seq Scan	supermarket_analysis	0.00 - 3451.00	1	31.092	((siparis_id + 0) = 85432)

3. Sorgu İyileştirme ve Uzun Süren Sorguların Optimizasyonu

Sistemde darboğaz yaratan sorguları tespit etmek için Execution Plan (Çalışma Planı) analizleri yapılmıştır. Analiz sonucunda, WHERE şartında gereksiz matematiksel manipülasyon (siparis_id + 0 = 85432) içeren sorguların, indeks kullanımını engelleyerek veritabanını "Tam Tablo Taraması"na (Seq Scan) zorladığı ve sorgu süresini uzattığı tespit edilmiştir.

```
SELECT * FROM supermarket_analysis WHERE siparis_id = 85432;
```

Execution plan - 1

Node Type	Entity	Cost	Rows	Time	Condition
Index Scan	supermarket_analysis	0.29 - 8.31	1	0.795	(siparis_id = 85432)

Bu uzun süren sorgu analiz edildikten sonra optimize edilmiştir. Matematiksel manipülasyon kaldırılarak (siparis_id = 85432 şeklinde) sorgu (indeks kullanabilir) hale getirilmiştir. Optimizasyon sonucunda sistemin Index Scan (İndeks Taraması) yapması sağlanmış ve sorgu maliyeti (cost) milisaniyeler seviyesine indirilerek performans artışı kanıtlanmıştır.

4. İndeks Yönetimi (Gereksiz İndekslerin Kaldırılması)

Performansı artırmak için oluşturulan indekslerin sistemdeki kullanım oranları incelenmiştir. Her indeksin diskte ekstra maliyet yarattığı göz önünde bulundurularak,

sistem görünümleri üzerinden (pg_stat_user_indexes) arka planda hiç kullanılmayan indeksler tespit edilmiştir.

```
SELECT
    relname AS tablo_adi,
    indexrelname AS indeks_adi,
    idx_scan AS kullanim_sayisi
FROM pg_stat_user_indexes
WHERE relname = 'supermarket_analysis';
```

pg_stat_user_indexes 1 | Execution plan - 2 | Execution plan - 3

SELECT relname AS tablo_adi, indexrelnam | Enter a SQL expression to filter results (use Ctrl+

	tablo_adi	indeks_adi	kullanim_sayisi
1	supermarket_analysis	supermarket_analysis_pkey	1
2	supermarket_analysis	idx_silinecek_test_indeksi	0

Sistem kaynaklarını tüketen bu gereksiz indeks, DROP INDEX komutu ile veritabanından başarıyla temizlenerek disk alanı ve veri manipülasyonu (DML) performansı korunmuştur.

```
DROP INDEX idx_silinecek_test_indeksi;
```

Statistics 1 | Execution plan - 2 | Execution plan - 3

Name	Value
Updated Rows	0
Execute time	0.008s
Start time	Mon Apr 20 22:11:47 TRT 2026
Finish time	Mon Apr 20 22:11:47 TRT 2026
Query	DROP INDEX idx_silinecek_test_indeksi

```sql
SELECT
    relname AS tablo_adi,
    indexrelname AS indeks_adi,
    idx_scan AS kullanim_sayisi
FROM pg_stat_user_indexes
WHERE relname = 'supermarket_analysis';
```

pg_stat_user_indexes 1 | Execution plan - 2 | Execution plan - 3

SELECT relname AS tablo_adi, indexrelnam | Enter a SQL expression to filter results (use

	tablo_adi	indeks_adi	kullanim_sayisi
1	supermarket_analysis	supermarket_analysis_pkey	1

5. Veri Yöneticisi Rolleri (Erişim Yönetimi)

Veri güvenliğini sağlamak ve "En Az Ayrıcalık Prensibini" uygulamak amacıyla Role-Based Access Control (RBAC) mekanizması kurulmuştur. Farklı kullanıcı senaryoları için erişim yönetimi yapılandırılmıştır:

- Sadece veri analizi ve raporlama yapacak kullanıcılar için salt-okunur yetkiye sahip rapor_uzmani rolü oluşturulmuştur.
- Tablo üzerinde veri manipülasyonu (INSERT, UPDATE, DELETE) yapacak yöneticiler için daha geniş yetkileri barındıran veri_yoneticisi rolü tanımlanmıştır.

```sql
CREATE ROLE rapor_uzmani WITH LOGIN PASSWORD 'sifre123';
GRANT SELECT ON supermarket_analysis TO rapor_uzmani;

CREATE ROLE veri_yoneticisi WITH LOGIN PASSWORD 'sifre456';
GRANT SELECT, INSERT, UPDATE, DELETE ON supermarket_analysis TO veri_yoneticisi;
```

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.037s
Start time	Mon Apr 20 21:53:53 TRT 2026
Finish time	Mon Apr 20 21:53:53 TRT 2026
Query	-- 1. Raporlama ekibi için sadece OKUMA (SELECT) yetkisi olan rol:
	CREATE ROLE rapor_uzmani WITH LOGIN PASSWORD 'sifre123';
	GRANT SELECT ON supermarket_analysis TO rapor_uzmani;
	-- 2. Veri giriş ekibi için tüm yetkilere sahip rol:
	CREATE ROLE veri_yoneticisi WITH LOGIN PASSWORD 'sifre456';
	GRANT SELECT, INSERT, UPDATE, DELETE ON supermarket_analysis TO veri_yoneticisi